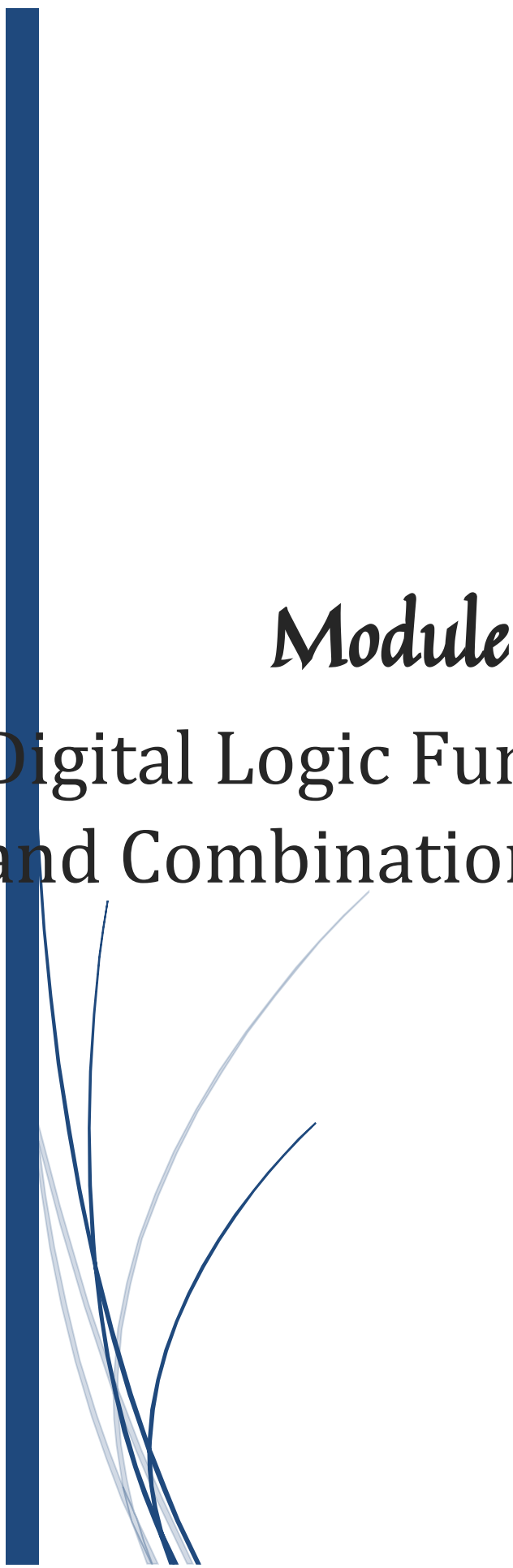
Module

Digital Logic Fun
and Combinatio



Syllabus

Boolean Algebra and Logic Circuits: Introduction, Number systems & Conversion: Decimal, Binary, Hexadecimal, Octal, Complement of Binary numbers, Boolean algebra theorems, Logic gates.

Combinational Circuits: Adders: Half adder and Full adder.

Activities: Demonstrate the working of basic gates and De Morgan's law using trainer kits.

Self-Learning Topics: Conversion: Octal to Hexadecimal and vice versa.

Textbook 2: 10.1, 10.2, 10.4, 10.5.1, 10.5.2

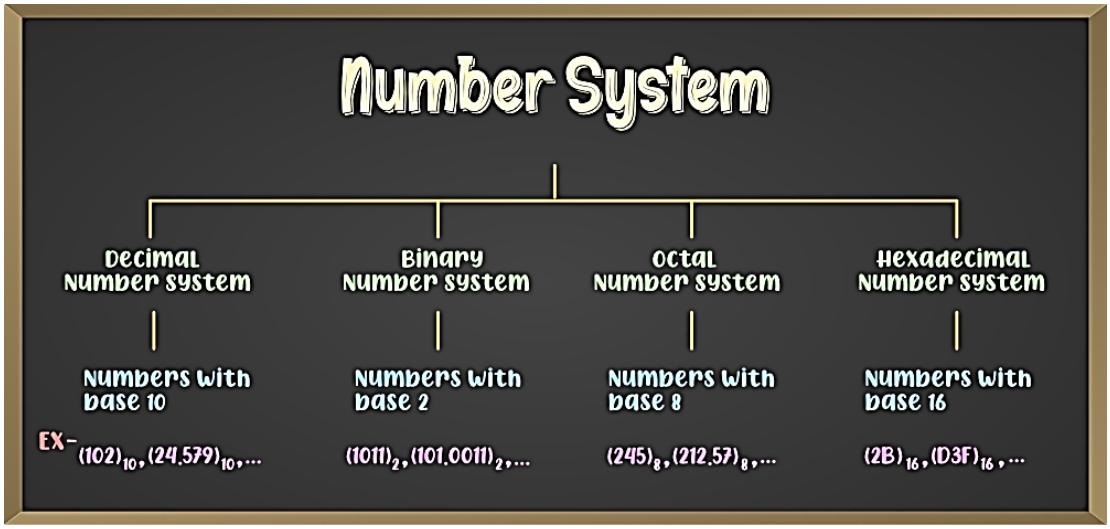
Textbook: Basic Electronics by D P Kothari and I J Nagrath, McGraw Hill Education (India), 2nd Edition, 2018.

Digital Electronics

Introduction

- **Digital** refers to processes carried out using **discrete units**.
- Digital numbers are used to represent **whole numbers** using individual units or groups.
- **Analog numbers** represent **continuous, directly measurable quantities** such as:
    - Voltage
    - Speed
    - Distance
- Analog values represent **real-world physical quantities**.
- Both **analog and digital methods** are used in computing.

Number Systems



1. Decimal Number System:

Combinations of ten digits from 0 to 9 are used to represent any number. In this system the next position to the left from the decimal point represents units, tens, hundreds, thousands etc. and the next position to the right after the decimal point represents () or 10^{-1}, () or 10^{-2}, () or 10^{-3} etc…

Digits are: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 and Base value: 10

Example:

1. $6841=6000+800+40+1 = 6 \times 10^3+8 \times 10^2 +4 \times 10^1+ 1 \times 10^0$
2. $123.34 = 1 \times 10^2+ 2 \times 10^1 + 3 \times 10^0 + 3 \times 10^{-1}+ 4 \times 10^{-2} =100 + 20 + 3 + 0.3 + 0.04$

2. Binary Number System:

In binary numbering systems only two digits 0 and 1 are used to represent any number. It will go like 0, 1, 10, 11, 100, 101, 110, 111, 1000 and so on. It is the numbering system used in computers. In this system the next position to the left from the binary point represents units, 2's, 4's, 8's etc. and the next position to the right after the binary point represents () or 2^{-1}, () or 2^{-2}, () or 2^{-3} etc…

Digits are: 0, 1 and **Base value:** 2

Example: $1010.101 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}$

$= 8 + 0 + 2 + 0 + 0.5 + 0 + 0.125 = 10.625$

3. Octal Number System:

In octal numbering system combination of eight digits from 0 to 7 are used to represent a number. It will go from 0…7, a two digit sequence is from 10..77 and a three digit sequence is from 100…777 and so on. It is used to shorten the binary numbers.

Digits are: 0, 1, 2, 3, 4, 5, 6, 7 and Base value: 8

Example: $376 = 3 \times 8^2 + 7 \times 8^1 + 6 \times 8^0 = 192 + 56 + 6 = 254$

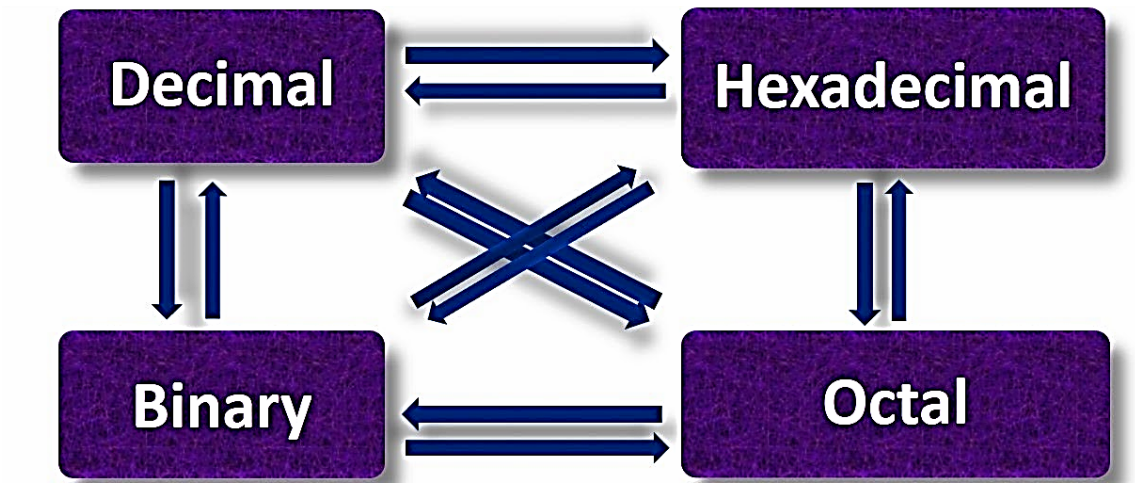
4. Hexadecimal Number System:

In hexadecimal numbering system combination of 16 digits from 0 to 9 and A to F are used to represent a number. It will go from 0…F, a two digit sequence is from 10…FF and a three digit sequence from 100…FFF and so on. It is used to represent data's and memory addresses.

Digits are: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F and **Base value:** 16

Example: $2A3B = 2 \times 16^3 + 10 \times 16^2 + 3 \times 16^1 + 11 \times 16^0 = 8192 + 2560 + 48 + 11 = 10811$

Conversions:

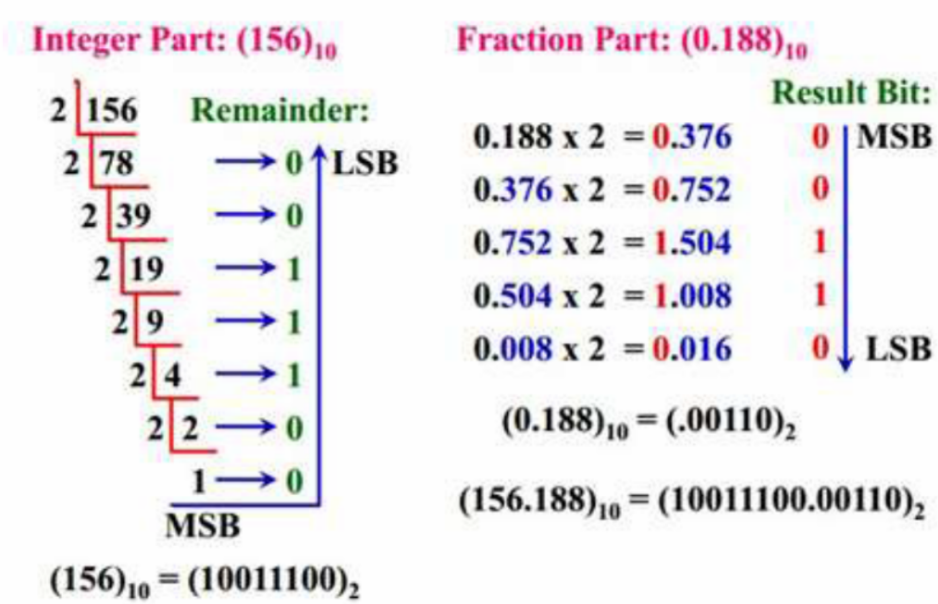


Decimal to Binary:

Integer Part: To convert the decimal integer to binary, divide the number by 2 and the successive quotients by 2. The successive remainders (which can be only 0 or 1) written in reverse order form the equivalent binary number.

Fractional Part: To convert decimal fractions to binary, multiply the number by 2 repeatedly until the fraction product is 0 (or until the desired number of binary places). The whole digits of the multiplication results produce the answer, with first as MSB and the last as LSB.

Example 1: Convert 156.188 decimal to binary.



Example 2: Convert 12.125 decimal to binary.
$(12)_{10} = (?)_2$
$12 \div 2 \Longrightarrow Q = 6, R = 0$
$6 \div 2 \Longrightarrow Q = 3, R = 0$
$3 \div 2 \Longrightarrow Q = 1, R = 1$
$(12)_{10} = (1100)_2$
$0.125 \times 2 = 0.25 \Longrightarrow$ Carry 0
$0.25 \times 2 = 0.5 \Longrightarrow$ Carry 0
$0.5 \times 2 = 1.0 \Longrightarrow$ Carry 1
$(0.122)_{10} = (0.001)_2$
Therefore, $(12.125)_{10} = (1100.001)_2$

Exercise:
1. Convert the following decimal number to binary up to four binary places.
(a) $(47.8125)_{10} = (\ )_2$
(b) $(100.974)_{10} = (\ )_2$
(c) $(29.3749)_{10} = (\ )_2$
(d) $(105.202)_{10} = (\ )_2$
(e) $(1024.625)_{10} = (\ )_2$
(f) $(555)_{10} = (\ )_2$
(g) $(0.825)_{10} = (\ )_2$

Answers

(a) $(47.8125)_{10} = (101111.1101)_2$

(b) $(100.974)_{10} \approx (1100100.11111001)_2$ *(approximate)*

(c) $(29.3749)_{10} \approx (11101.01011111)_2$ *(approximate)*

(d) $(105.202)_{10} \approx (1101001.00110011)_2$ *(approximate)*

(e) $(1024.625)_{10} = (10000000000.101)_2$

(f) $(555)_{10} = (1000101011)_2$

(g) $(0.825)_{10} \approx (0.11010011)_2$ *(approximate)*

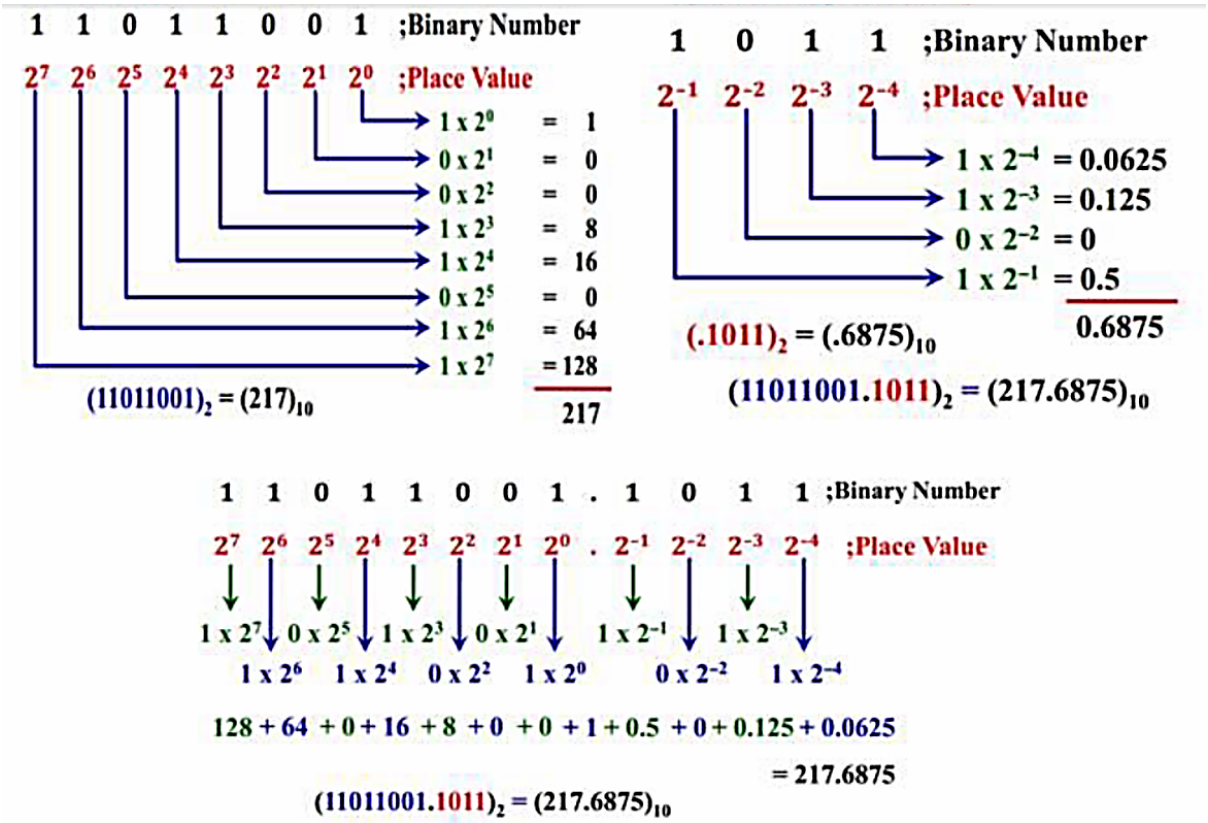
Binary to Decimal Conversion:

To convert the binary number to its equivalent decimal, multiply the binary digits by its corresponding weights, and then add them.

Weights from the left of binary point are: 2^0 , 2^1 , 2^2 , 2^3.etc

Weights from the right of binary point are: 2^{-1}, 2^{-2} , 2^{-3} , 2^{-4}, 2^{-5}.etc..

Example 1: Convert the binary number 11011001.1011 to decimal



Example 2: Convert $(1100.001)_2$ to decimal

$$(1100.001)_2 = 1x2^3 + 1x2^2 + 0x2^1 + 0x2^0 + 0x2^{-1} + 0x2^{-2} + 1x2^{-3} = 8 + 4 + \frac{1}{8}$$
$$= (12.125)_{10}$$

Exercise:

1. Convert the following binary number to equivalent decimal number.

(a) $(1101)_2 = (\ )_{10}$

(b) $(10101)_2 = (\ )_{10}$

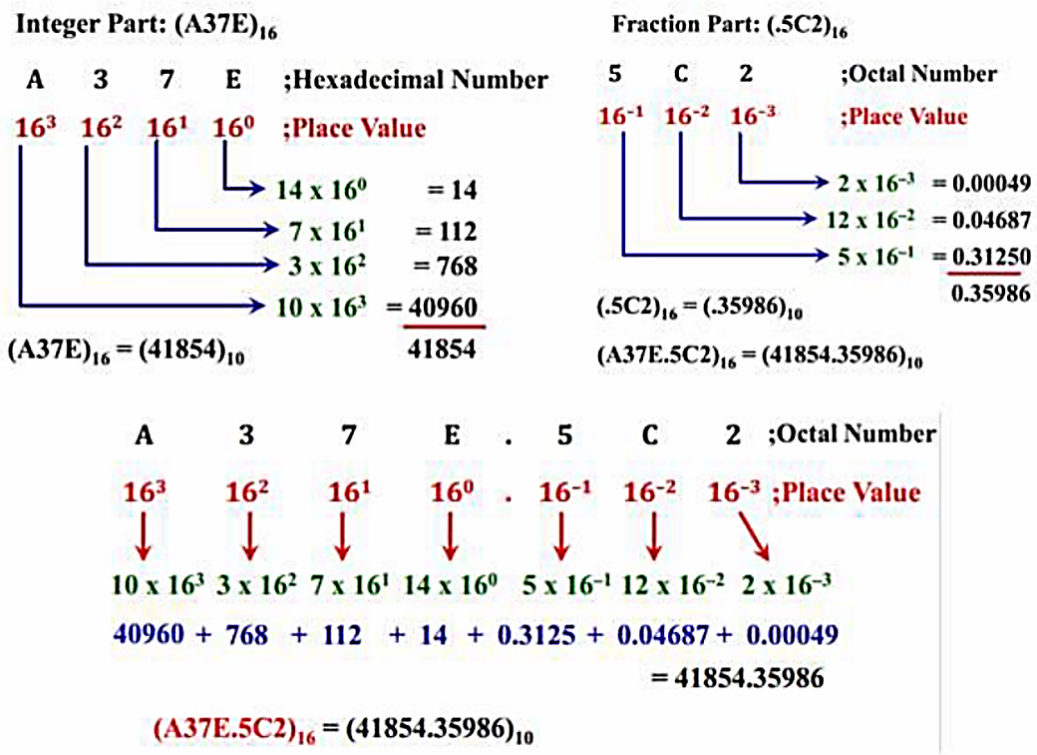
(c) $(11001.011)_2 = (\ )_{10}$

(d) $(11001.110)_2 = (\ )_{10}$

Hexadecimal to Decimal Conversion:

- To convert the hexadecimal number to its equivalent decimal, multiply the hexadecimal digits by its corresponding weights, and then add them.
- Weights from the left of hexadecimal point are: $16^0, 16^1, 16^2, 16^3$ etc.
- Weights from the right of hexadecimal point are :$16^{-1}, 16^{-2}, 16^{-3}$ etc.

Example 1: Convert the hexadecimal number $(A37E.5C2)_{16}$ to decimal.



Example 2: Convert the hexadecimal number $(3A1.4)_{16}$ to decimal.

$$(3A1.4)_{16} = 3x16^2 + 10x16^1 + 1x16^0 + 4x16^{-1} = 768 + 160 + 1 + \frac{4}{16} = (929.25)_{10}$$

Problems:

Convert the following hexadecimal number to equivalent decimal number.

(a) $(AF)_{16} = (\ )_{10}$

(b) $(B6A)_{16} = (\ )_{10}$

(c) $(ABC.CD)_{16} = (\ )_{10}$

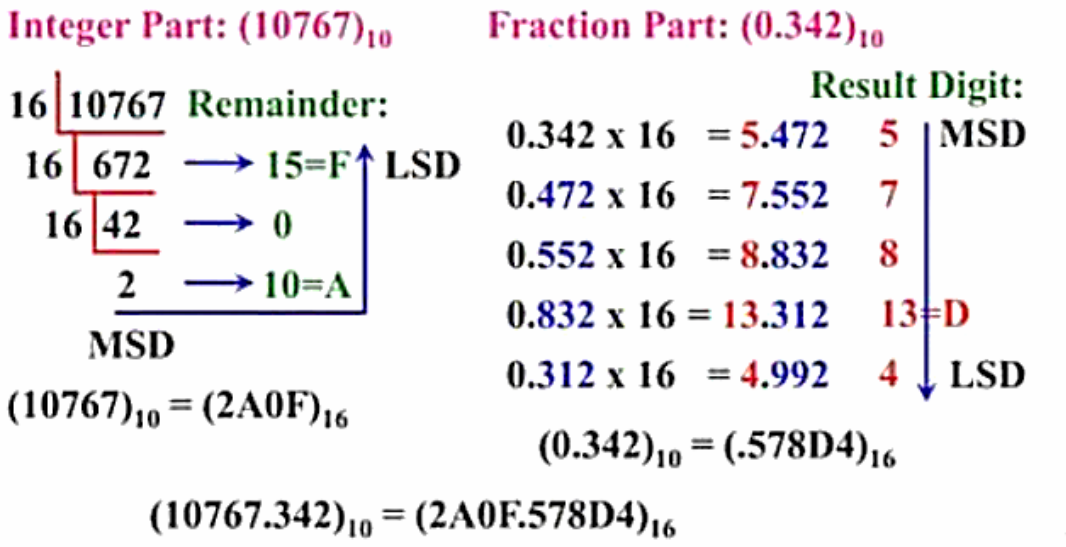
(d) $(9FC1)_{16} = (\ )_{10}$

Decimal to Hexadecimal Conversion:

Integer Part: To convert a decimal integer to hexadecimal, divide the number by 16 and the successive quotients by 16. The successive remainders (which can be only 0 to 9 or A to F) written in reverse order form the equivalent hexadecimal number.

Fractional Part: To convert decimal fractions to hexadecimal, multiply the number by repeatedly until the fraction product is 0 (or until the desired number of hexadecimal places). The whole digits of the multiplication results produce the answer, with first as MSB and the last as LSB.

Example 1: Convert $(10767.342)_{10}$ to hexadecimal.



Example 2: Convert $(5386.345)_{10}$ to hexadecimal.

$5386 \div 16 \Longrightarrow Q = 336, R = 10\ (A)$
$336 \div 16 \Longrightarrow Q = 21, R = 0$
$21 \div 16 \Longrightarrow Q = 1, R = 5$
$(5386)_{10} = (150A)_{16}$

$0.345 \times 16 = 5.52 \Longrightarrow$ Carry 5
$0.52 \times 16 = 8.32 \Longrightarrow$ Carry 8
$0.32 \times 16 = 5.12 \Longrightarrow$ Carry 5
$0.12 \times 16 = 1.92 \Longrightarrow$ Carry 1
$(0.345)_{10} = (0.5851)_{16}$
Therefore, $(5386.345)_{10} = (150A.5851)_{16}$

Problems:

Convert the following decimal number to hexadecimal.

(a) $(57345)_{10} = (\ )_{16}$

(b) $(342.56)_{10} = (\ )_{16}$

Binary to Octal Conversion:

Binary → Octal conversion is done by grouping binary digits into sets of 3 (from right to left)

Steps:

1. Start from the rightmost bit of the binary number.
2. Group digits into sets of three.
3. If needed, add leading zeros to complete a group.
4. Convert each group into its octal equivalent.

Example :

(a) $101101_2 = ()_8$

$101\ 101 = (55)_8$

(b) $1100110_2 = ()_8$

$001\ 100\ 110 = (146)_8$

Binary to Hexadecimal Conversion:

Binary → Hexadecimal conversion is done by grouping binary digits into sets of 4 (from right to left)

Steps:

1. Start from the rightmost bit of the binary number.

2. Group digits into sets of four.
3. If needed, add leading zeros to complete a group.
4. Convert each group into its octal equivalent.

Example:

(a) $10111101_2 = ()_{16}$

1011 1101 = $(BD)_{16}$

(b) $1110010_2 = ()_{16}$

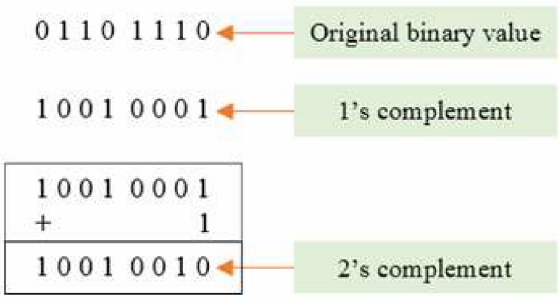
0111 0010 = $(72)_8$

Complement of Binary Numbers:

There are two types of complements for binary numbers and are used to perform subtraction using addition.

i. 1's complement: Obtained by changing all 0's to 1's and all 1's to 0's.

ii. 2's complement: Obtained by adding 1 to 1's complement of a number.



Binary Addition:

The rules of binary addition are:

0 + 0 = 0

0 + 1 = 1

1 + 0 = 1

1 + 1 = 0 (with carry 1)

Example 1: Perform addition of $(11001100)_2$ and $(11011010)_2$

1 1 1 ⟵ Carry

1 1 0 0 1 1 0 0

1 1 0 1 1 0 1 0

1 1 0 1 0 0 1 1 0

Example 2: Add $(28)_{10}$ and $(15)_{10}$ by converting them into binary.

1 1 ⟵ Carry

$(28)_{10}$ = 1 1 1 0 0

$(15)_{10}$ = 0 1 1 1 1

1 0 1 0 1 1

Example:

1.
101 +
010
111

2.
110 +
001
111

3.
111 +
001
1000

4.
1001 +
0010
1011

5.
1010 +
0101
1111

6.
1110 +
0111
10101

7.
1011 +
1101
11000

8.
1111 +
1111
11110

9.
101101 +
110011
1100000

10.
111000 +
101111
1100111

11.
1001111 +
0110110
10000101

12.
1111111 +
0000001
10000000

13.
1010101 +
0101010
1111111

14.
1001100110 +
0110011001
1111111111

Binary Subtraction:

The rules of binary subtraction are:

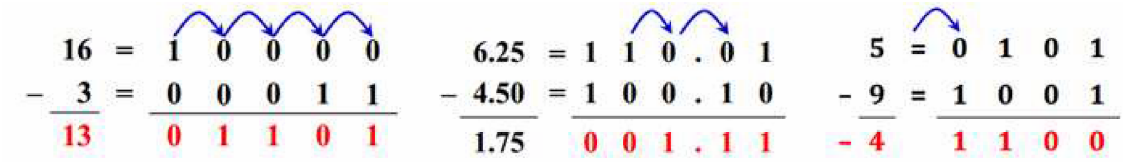
0 – 0 = 0

1 – 0 = 1

1 – 1 = 0

0 – 1 = 1 (Taking barrow from outside)

Examples:



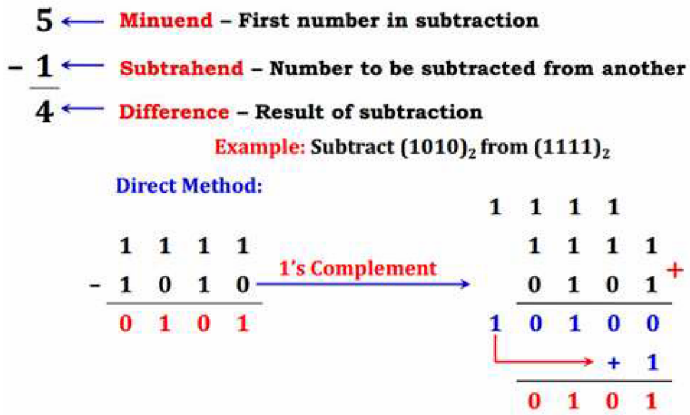
Binary Subtraction Using 1's Complement:(m-n)

1. Take 1's complement of subtrahend (n) and add it to minuend(m).

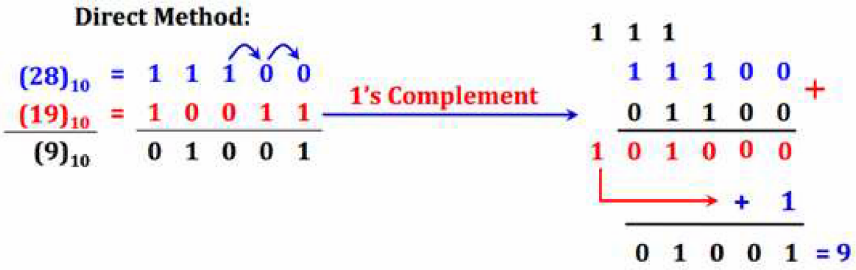
2. If carry is generated, then result is positive and in the true form. Add carry to the result to get final

result.

3. If carry is not generated, then result is negative and in 1's complements form.



Example 1: Perform $(28)_{10} - (19)_{10}$ using 1's complement.



Example 2: Perform $(15)_{10} - (28)_{10}$ using 1's complement.



- **No carry, hence result is negative in 1's complement form.**

Original Result: 1 0 0 1 0

1's Complement of Result: 0 1 1 0 1 = 13

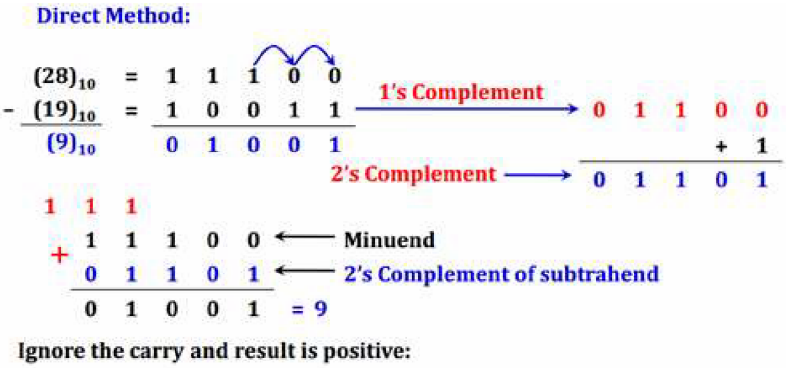
Binary Subtraction Using 2's Complement (m-n):

1. Take 2's complement of subtrahend (n) and add it to minuend (m).

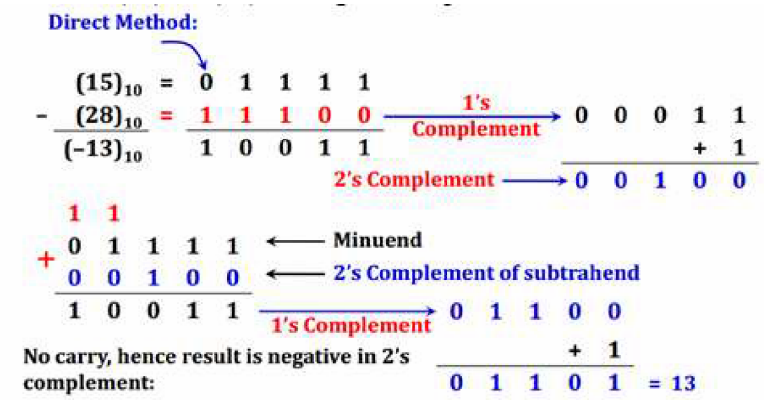
2. If carry is generated, then result is positive and discard the carry.

3. If carry is not generated, then result is negative and in 2's complement form.

Example 1: Perform $(28)_{10} - (19)_{10}$ using 2's complement.



Example 3: Perform $(15)_{10} - (28)_{10}$ using 2's complement.



Examples:

1) 101 - 010

1's Complement:

010 → 101

101 + 101 = 1010 → add carry → 011

2's Complement:

010 → 101 +1 = 110

101 + 110 = 1011 → ignore carry → 011

2) 110 - 001

1's Complement:

001→110

110+110=1100 → +1 = 101

2's Complement:

001→111

110+111=1101 → ignore carry = 101

3) 1000 - 0011
1's Complement:
0011→1100
1000+1100=10100 → +1 = 0101

2's Complement:
0011→1101
1000+1101=10101 → ignore carry = 0101

4) 1110 - 0101
1's Complement:
0101→1010
1110+1010=11000 → +1 = 1001

2's Complement:
0101→1011
1110+1011=11001 → ignore carry = 1001

5) 10101 - 01111
1's Complement:
01111→10000
10101+10000=100101 → +1 = 00110

2's Complement:
01111→10001
10101+10001=100110 → ignore carry = 00110

6) 11001 - 00111
1's Complement:
00111→11000
11001+11000=110001 → +1 = 10010

2's Complement:
00111→11001
11001+11001=110010 → ignore carry = 10010

7) 11111 - 01010
1's Complement:
01010→10101
11111+10101=110100 → +1 = 10101

2's Complement:

01010→10110
11111+10110=110101 → ignore carry = 10101

8) 100000 - 000001
1's Complement:
000001→111110
100000+111110=1011110 → +1 = 11111

2's Complement:
000001→111111
100000+111111=1011111 → ignore carry = 11111

9) 101010 - 010101
1's Complement:
010101→101010
101010+101010=1010100 → +1 = 010101

2's Complement:
010101→101011
101010+101011=1010101 → ignore carry = 010101

Boolean algebra theorems:

Boolean algebra is a mathematical system used in digital electronics and logic design. It consists of binary operations such as AND (.), OR (+), and the unary NOT operation. The variables can take only two values: 0 and 1.

Basic Logic Operations

AND Operation

Symbol: $Y = A \cdot B$
The output is 1 only when both inputs A and B are 1.

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

OR Operation

Symbol: $Y = A + B$
The output is 1 if either A or B or both are 1.

A	B	Y

0	0	0
0	1	1
1	0	1
1	1	1

NOT Operation

Symbol: $Y = \bar{A}$
The NOT operation produces the complement of the input.

A	Y
0	1
1	0

Special Cases:

- AND Operation:
  - $A \cdot 0 = 0$
  - $A \cdot 1 = A$
  - $A \cdot A = A$
  - $A \cdot \bar{A} = 0$
- OR Operation:
  - $A + 0 = A$
  - $A + 1 = 1$
  - $A + A = A$
  - $A + \bar{A} = 1$

Boolean Algebra Laws:

Law	AND Form	OR Form
Identity Law	$1 \cdot A = A$	$0 + A = A$
Null Law	$0 \cdot A = 0$	$1 + A = 1$
Idempotent Law	$A \cdot A = A$	$A + A = A$
Inverse Law	$A \cdot \bar{A} = 0$	$A + \bar{A} = 1$
Commutative Law	$A \cdot B = B \cdot A$	$A + B = B + A$
Associative Law	$(A \cdot B) \cdot C = A \cdot (B \cdot C)$	$(A+B)+C = A+(B+C)$
Distributive Law	$A+BC = (A+B)(A+C)$	$A(B+C)=AB+AC$
Absorption Law	$A(A+B) = A$	$A+AB=A$
De Morgan's Law	$(\,) = \bar{A} + \bar{B}$	$(\,) = \bar{A} \cdot \bar{B}$

Proof:

Distributive Law:
(A + B)(A + C) = AA + AC + BA + BC
= A + A(B + C) + BC
= A[1 + (B + C)] + BC
= A + BC
Absorption Law Proof
A(A + B) = AA + AB
= A + AB
= A(1 + B)
= A
De Morgan's Theorem
1. ( ) = $\bar{A} \cdot \bar{B}$
2. ( )= $\bar{A} + \bar{B}$

Rules:

- Change all variables to complements.
- Change all AND operation to ORs.
- Change all OR operation to ANDs.
- Take the complement the entire expression.

A	B	$\bar{A}$	$\bar{B}$	$\bar{A} \cdot \bar{B}$	$\bar{A} + \bar{B}$	( )	( )
0	0	1	1	1	1	1	1
0	1	1	0	0	1	0	1
1	0	0	1	0	1	0	1
1	1	0	0	0	0	0	0

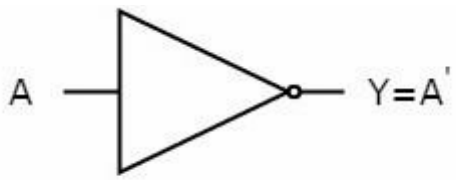
Logic gates:

Logic Levels
Logic 0 represents LOW voltage and Logic 1 represents HIGH voltage.

NOT Gate:
The NOT gate or inverter produces the complement of the input.

Boolean Expression: Y = $\bar{A}$

Symbol:



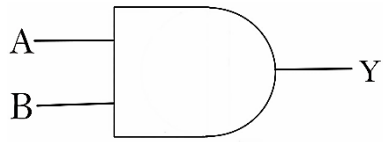
Truth table:

A	Y
0	1
1	0

AND Gate:
The AND gate performs logical multiplication. Output is HIGH only when all inputs are HIGH.

Boolean Expression: $Y = A \cdot B$

Symbol:



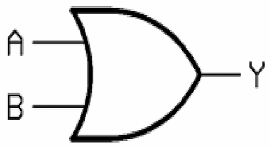
Truth table:

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

OR Gate:
The OR gate performs logical addition. Output is HIGH if any input is HIGH.

Boolean Expression: $Y = A + B$

Symbol:



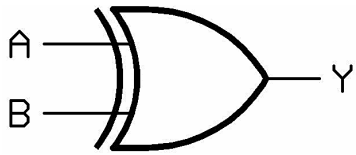
Truth table:

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

XOR Gate:
Exclusive OR gate gives output HIGH when inputs are different.

Boolean Expression: $Y = A \oplus B$

Symbol:



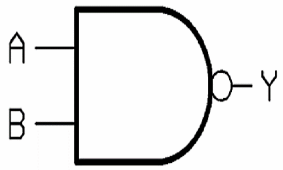
Truth table:

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

NAND Gate:
NAND gate is the complement of AND gate.

Boolean Expression: Y =

Symbol:



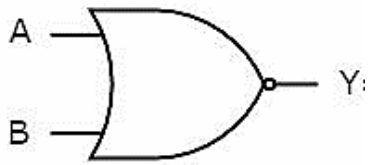
Truth table:

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

NOR Gate:
NOR gate is the complement of OR gate.

Boolean Expression: Y = ( )

Symbol:



Truth table:

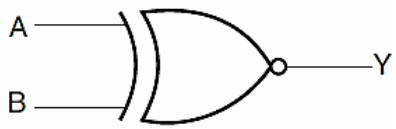
A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

XNOR Gate:

XNOR gate gives HIGH output when both inputs are equal.

Boolean Expression: Y = A ⊙ B

Symbol:



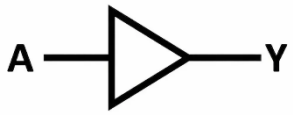
Truth table:

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

Buffer Gate:
A buffer produces the same output as input and is used to increase fan-out.

Boolean Expression: Y = A

Symbol:



Truth table:

A	Y
0	0
1	1

Universal Gates:
NAND and NOR gates are called universal gates because any logic function can be implemented using only NAND gates or only NOR gates.

Combinational Circuits:

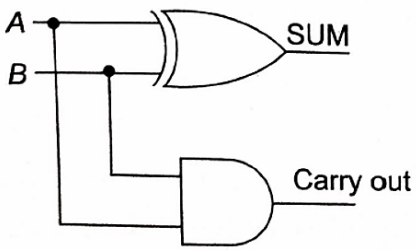
Half Adder:
A half adder adds two binary bits and produces Sum and Carry.

Truth table:

A	B	Sum	Carry
0	0	0	0

0	1	1	0
1	0	1	0
1	1	0	1

Equations:
$S = A \oplus B$
$C = A \cdot B$



Full Adder:
A Full Adder is a combinational logic circuit used to add three binary bits:

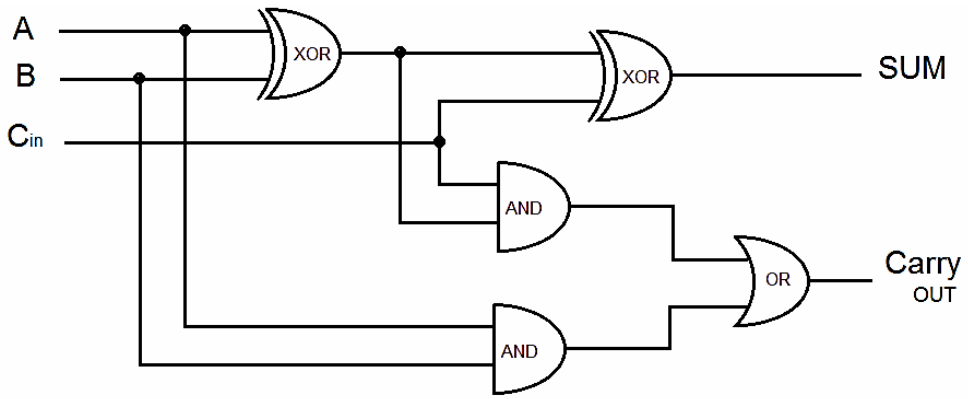
- Two significant bits: A and B
- One carry input: C_{in}

It produces:

- Sum output (S)
- Carry output (C_{out})

Truth table:

A	**B**	**C_{in}**	**S**	**C_{out}**
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



Equation:

$S=A \oplus B \oplus C_{in}$

$C_{out}=AB+BC_{in}+AC_{in}$